# Introduction

- Virtualization vs. Abstraction
- Emulation vs. Simulation
- Runtime System
- Virtual Machine

# Virtualization vs. Abstraction

## Abstraction

*Abstraction* neglects non-essential details, i.e., a reduction to the important aspects is performed. In technical systems, there is often a hierarchy of abstractions with several levels. The connection between different levels of abstraction is made by *well-defined interfaces*.

Examples:

# Virtualization vs. Abstraction

## Abstraction

*Abstraction* neglects non-essential details, i.e., a reduction to the important aspects is performed. In technical systems, there is often a hierarchy of abstractions with several levels. The connection between different levels of abstraction is made by *well-defined interfaces*.

Examples:

- SATA disk controller $\rightarrow$ AHCI interface $\rightarrow$ block device interface in the kernel $\rightarrow$ file I/O interface in the kernel $\rightarrow$ file I/O interface in the C runtime system $\rightarrow$ `java.io.FileOutputStream` $\rightarrow$ `java.io.OutputStream`
- OSI model for network communication

# Virtualization vs. Abstraction

## Virtualization

A system $G$ (guest) is *virtualized* in a system $H$ (host), when for every operation (= state transformation) $\sigma$ on $G$, there exists a corresponding operation $\sigma'$ on $H$.

# Virtualization vs. Abstraction

## Virtualization

A system $G$ (guest) is *virtualized* in a system $H$ (host), when for every operation (= state transformation) $\sigma$ on $G$, there exists a corresponding operation $\sigma'$ on $H$.
Formally: Let $\mathcal{S}(G)$ and $\mathcal{S}(H)$ be the sets of states of $G$ and $H$, respectively. An injective map $V : \mathcal{S}(G) \to \mathcal{S}(H)$ *virtualizes* $G$ in $H$, when for each operation $\sigma : \mathcal{S}(G) \to \mathcal{S}(G)$ on $G$, there exists a corresponding operation $\sigma' : \mathcal{S}(H) \to \mathcal{S}(H)$ on $H$ and $V \circ \sigma = \sigma' \circ V$ holds.

# Virtualization vs. Abstraction

## Virtualization

A system $G$ (guest) is *virtualized* in a system $H$ (host), when for every operation (= state transformation) $\sigma$ on $G$, there exists a corresponding operation $\sigma'$ on $H$.
Formally: Let $\mathcal{S}(G)$ and $\mathcal{S}(H)$ be the sets of states of $G$ and $H$, respectively. An injective map $V : \mathcal{S}(G) \to \mathcal{S}(H)$ *virtualizes* $G$ in $H$, when for each operation $\sigma : \mathcal{S}(G) \to \mathcal{S}(G)$ on $G$, there exists a corresponding operation $\sigma' : \mathcal{S}(H) \to \mathcal{S}(H)$ on $H$ and $V \circ \sigma = \sigma' \circ V$ holds.

$V$ must be injective to ensure that two different states in the guest system $G$ are represented by two different states in the host system $H$; otherwise, $H$ would be an abstraction of $G$ and the correspondence between states in $H$ and $G$ would get lost.

# Example

## Java byte code

| | |
|---|---|
| `iadd` | compute the sum of two values in the operand stack and put the result on the stack |

## x86 assembly

| | |
|---|---|
| `popl %eax` | fetch first operand |
| `popl %ebx` | fetch second operand |
| `addl %eax,%ebx` | add operands |
| `pushl %eax` | put sum on the stack |

# Example

## Java byte code

| | |
|---|---|
| `iadd` | compute the sum of two values in the operand stack and put the result on the stack |

## x86 assembly

| | |
|---|---|
| `popl %eax` | fetch first operand |
| `popl %ebx` | fetch second operand |
| `addl %eax,%ebx` | add operands |
| `pushl %eax` | put sum on the stack |

Abstraction: (implicit) stack structure in Java byte code instead of stack and registers in x86 code.

Virtualization: the state transition expressed by the `iadd` instruction corresponds to the state transition expressed by the four x86 instructions.

Emulation: the behavior of the `iadd` instruction is recreated in x86 assembly.

# Emulation vs. Simulation

## Emulation

*Emulation* recreates the observable behavior (semantics) of a system in another system. The internals of the emulated system need not be reproduced in detail.

## Simulation

Simulation goes beyound emulation by recreating all the internal processes of a system, not only the results of a computation.

# Emulation vs. Simulation

## Emulation

*Emulation* recreates the observable behavior (semantics) of a system in another system. The internals of the emulated system need not be reproduced in detail.

## Simulation

Simulation goes beyound emulation by recreating all the internal processes of a system, not only the results of a computation.

Example: The simulation of a microprocessor encompasses the exact reproduction of the clock cycles each instruction takes or the cache behavior. An emulation reproduces only the effects of the instructions on the memory and registers (neglecting, e.g., the exact timing).

# Virtualization

The definition of virtualization allows for *several* similar guest systems to be virtualized within the same host system.

The guest systems are often resources (together with their interfaces). Virtualization then lets it appear as if there were several instances of the resource that can be used independently from each other.

Examples:

# Virtualization

The definition of virtualization allows for *several* similar guest systems to be virtualized within the same host system.

The guest systems are often resources (together with their interfaces). Virtualization then lets it appear as if there were several instances of the resource that can be used independently from each other.

Examples:

- threads and processes (virtualize the CPU)
- virtual memory (virtualizes physical RAM)
- file systems (virtualize storage media)
- windows and display managers (virtualize graphics hardware)
- network sockets (virtualizes network hardware)

# Runtime System

A runtime system supports the execution of a program by providing various services at run time. These services can be used by a process explicitly (by calling an API function, e.g., for memory allocation) or implicitly (without any direct action, e.g., thread scheduling).

Usually, the runtime system provides an abstraction from the operating system (e.g., the C runtime system abstracts from Linux/BSD/Solaris/Windows/...). The abstraction is often incomplete, i.e., operating system specific differences remain. Example: `man 2 signal`

A runtime system commonly virtualizes resources such as CPU time, memory, network, etc. The dividing line between operating system and runtime system is not always clearly defined.

# Virtual Machine

A runtime system defines an API (application programming interface), i.e., an interface with a set of operations (= functions) that can be called. There is compatibility on the source code level, i.e., execution on a different platform requires a re-compilation.

# Virtual Machine

A runtime system defines an API (application programming interface), i.e., an interface with a set of operations (= functions) that can be called. There is compatibility on the source code level, i.e., execution on a different platform requires a re-compilation.

## Virtual Machine

A *virtual machine* does not only define an API but also the instructions a program can execute.

# Virtual Machine

A runtime system defines an API (application programming interface), i.e., an interface with a set of operations (= functions) that can be called. There is compatibility on the source code level, i.e., execution on a different platform requires a re-compilation.

## Virtual Machine

A *virtual machine* does not only define an API but also the instructions a program can execute.

Virtual machines can abstract from the instruction set of a real machine (e.g., the Java virtual machine does that) but that is not necessarily so.

Depending on the type of virtual machine, the instruction set is an ABI (application binary interface) or an ISA (instruction set architecture) – discussed in more detail later.

Examples:

- Java: API = Java class library; instruction set = Java byte code
- QEMU, VMware, VirtualBox, etc.: instruction set = x86/x86_64 instructions; API $\hat{=}$ emulated hardware interfaces